

Arrays and the Simplest Useful ADT

Week 3 — Array Addresses, the Stack, and Two Things Stacks Do

Dr. Auqib Hamid Lone

PCC CS-301: Data Structures
GCET, Ganderbal

Week 3

Today's Three Questions

1. How does the machine find $A[i]$ without scanning for it?
2. How do we describe a “last in, first out” store before coding it?
3. What do brackets and postfix expressions have to do with that store?

Plan

First lay out memory in a row. Then name the stack contract. Then use one stack to check brackets and evaluate postfix.

The Calculator Needs Two Things

Our running example

The expression processor reads $(a+b)*c$: it stores operands somewhere, and it must check the brackets and evaluate the expression.

- ▶ Week 1 gave it heap blocks; Week 2 gave it a way to price lookups.
- ▶ Operands arrive in order — we want the i -th one *instantly*.
- ▶ Brackets nest — the *last* opened bracket must close *first*.

One need is positional (arrays); the other is nested (stacks).

Positional vs. Nested Needs

- ▶ **Positional**: “give me element number i ” — marks list, pixel row, operand table.
- ▶ **Nested**: “undo the most recent thing first” — brackets, function calls, backtracking.
- ▶ A linked walk gives neither instantly; a row of cells gives the first, and a disciplined pointer over that row gives the second.

The deal

Arrays buy $O(1)$ worst-case access by laying elements side by side; a stack buys nesting discipline with one index, `top`.

Where This Week Sits

- ▶ Week 2 ended with: linear scan $O(n)$ worst case vs. binary search $O(\log n)$ worst case — *if* the table supports instant access.
- ▶ This week builds the structure that makes “instant access” true.
- ▶ Week 4 then spends the stack on full expression conversion, Hanoi, and recursion.

The arc: state the contract (stack ADT), implement it once over an array, analyse what it costs — the same shape every week repeats.

First: the row of cells and how the machine addresses it.

Section 1

One-Dimensional Arrays

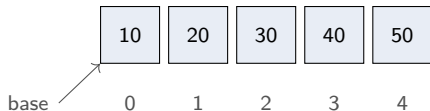
What Is an Array?

Definition

An array holds a fixed number of elements of one type in contiguous memory, reached by index — the standard treatment (see Horowitz & Sahni Ch. 2; Wirth §1.5) [2, 5].

- ▶ **Contiguous**: element $i + 1$ starts where element i ends.
- ▶ **0-based** in C: $A[0]$ is first, $A[n-1]$ is last.
- ▶ n counts the elements of A — our Week 1–2 convention.

A Row of Five Cells



- ▶ Five `int` cells, side by side; index i names the i -th cell.
- ▶ `base` is the address of `A[0]` — everything else is arithmetic from there.

Finding $A[i]$ Is Arithmetic

1D address formula

If each element takes w bytes,

$$\text{addr}(A[i]) = \text{base} + i \times w.$$

- ▶ No scan, no search — one multiply and one add.
- ▶ That is why access is $O(1)$ worst case: the work does not grow with n .

Worked example

$\text{base} = 2000$, $w = 4$: $A[3]$ lives at $2000 + 3 \times 4 = 2012$.

Bounds Are the Contract You Enforce

Valid range

For `int A[n]`, only `A[0..n-1]` exists. C does not check — reading `A[n]` is undefined behaviour.

- ▶ **Off by one**: looping `i <= n` instead of `i < n`.
- ▶ **Unchecked index**: trusting user input as an index directly.

Every array access on this course carries an explicit range check in the reasoning, even when C omits it at run time.

What Arrays Cost

Priced with Week 2's vocabulary

Name the operation, give the bound, state the case.

- ▶ **Access** $A[i]$: $O(1)$ worst case — the formula above.
- ▶ **Search** (unsorted): $O(n)$ worst case — nothing beats a scan.
- ▶ **Insert/delete** at position i : $O(n)$ worst case — up to $n - i$ elements shift.

Fast to read, slow to splice in the middle — the deficiency Week 6's linked lists will answer.

Check Your Prediction

An array holds $n = 1000$ marks; insert one at position 0

How many elements move?

1. None — the formula makes room.
2. One — only the neighbour shifts.
3. About 1000 — everything shifts one slot right.

Turn and talk

Why does the *position* of the insert decide the cost, when access cost never depends on position?

One dimension is a row. Two dimensions are rows of rows.

Section 2

Two-Dimensional Arrays

A 2D Array Is an Array of Arrays

C's view (row-major)

`int A[r][c]` is r rows, each a 1D array of c elements; row 0 sits first in memory, then row 1, and so on.

- ▶ Declaration says the shape: r rows, c columns.
- ▶ Element `A[i][j]`: row i , column j , both 0-based.
- ▶ Memory still holds one flat row of $r \times c$ cells — the mapping is pure arithmetic.

Row-Major Layout: 2×3

A[0][0] $k = 0$	A[0][1] $k = 1$	A[0][2] $k = 2$
A[1][0] $k = 3$	A[1][1] $k = 4$	A[1][2] $k = 5$

- ▶ Row 0 occupies flat slots $k = 0, 1, 2$; row 1 takes $k = 3, 4, 5$.
- ▶ Flat position: $k = i \times c + j$ — skip i whole rows, then j cells.

Row-Major Address Formula

Row-major address ($r \times c$, w bytes per element)

$$\text{addr}(A[i][j]) = \text{base} + ((i \times c) + j) \times w.$$

Worked example

3×4 `int` array, `base = 1000`, $w = 4$: `A[1][2]` is at $1000 + ((1 \times 4) + 2) \times 4 = 1024$.

C, C++, and Python (NumPy by default) all lay rows out this way.

Column-Major: the Transposed Twin

Column-major address ($r \times c$, w bytes per element)

$$\text{addr}(A[i][j]) = \text{base} + ((j \times r) + i) \times w.$$

- ▶ Column 0 sits first in memory, then column 1 — rows are interleaved, not contiguous.
- ▶ Same 3×4 example: $A[1][2]$ is at $1000 + ((2 \times 3) + 1) \times 4 = 1028$.
- ▶ Fortran and MATLAB use this order; mixing the two layouts up is a classic porting bug.

Row-major skips rows ($i \times c$); column-major skips columns ($j \times r$).

Check Your Prediction

A 3×4 array, row-major, base = 1000, $w = 4$

Where is $A[2][0]$?

1. $1000 + 2 \times 4 = 1008$.
2. $1000 + 8 \times 4 = 1032$.
3. $1000 + 6 \times 4 = 1024$.

Two full rows of 4 precede it: $k = 2 \times 4 + 0 = 8$.

Rows give us storage. Now the discipline for nested work.

Section 3

The Stack ADT

What Is a Stack?

Definition (LIFO)

A stack is a collection from which we remove the most recently added item first — last in, first out. One of Sedgewick's three collection types (p.120) [4].

- ▶ Think of a pile of trays: you add to the top and take from the top.
- ▶ The contract says nothing about arrays yet — arrays come next.

Stacks, queues, and bags differ only in which item leaves next (Sedgewick, p.120).

The Stack Contract

Stack ADT

Values plus three operations (Karumanchi Ch. 4, pp.163–204 PDF) [3]:

1. `push(x)` — place `x` on the top.
2. `pop()` — remove and return the top item; an error if empty.
3. `peek()` — return the top item without removing it.

Week 1's shape again: name the operations first, then implement them.

Implementing the Stack over an Array

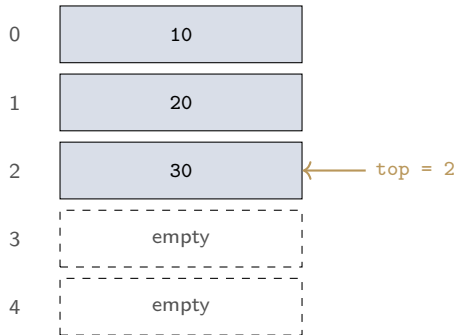
One array, one index

```
#define MAX 100
int stack[MAX];
int top = -1;    /* -1 means empty */
```

```
void push(int x) { if (top + 1 < MAX) stack[++top] = x; }
int pop(void)   { if (top >= 0) return stack[top--]; return -1; }
int peek(void)  { if (top >= 0) return stack[top]; return -1; }
```

Production code would signal errors explicitly; the -1 sentinel here keeps the slide readable.

Seeing top Move



- ▶ `push(40)` writes cell 3 and moves `top` to 3.
- ▶ `pop()` returns 30 and moves `top` back to 1.

Two Errors and the Cost

- ▶ **Overflow**: push onto a full stack ($\text{top} + 1 == \text{MAX}$).
- ▶ **Underflow**: pop or peek on an empty stack ($\text{top} == -1$).

Cost of each stack operation

push, pop, peek are each $O(1)$ worst case — one array access plus one index update.

Fixed capacity is the price: when the array fills, this implementation must refuse or grow (Week 6 removes the ceiling with links).

Check Your Prediction

After `push(10)`, `push(20)`, `pop()`, `push(30)` — what is `top`, and what does `peek()` return?

1. `top = 1`, `peek()` returns 30.
2. `top = 2`, `peek()` returns 20.
3. `top = 1`, `peek()` returns 20.

Trace the index, not just the values: $0 \rightarrow 1 \rightarrow 0 \rightarrow 1$.

The contract is set. Now watch one stack do real work.

Section 4

Two Things Stacks Do

Application 1: Are the Brackets Balanced?

Problem (Lab P1)

Given a string of brackets () [] { }, decide whether every opener is closed, in the right order.

- ▶ ([]) — nested correctly.
- ▶ ([]) — closed in the wrong order.
- ▶ ((()) — an opener never closed.

The last-opened bracket must close first — exactly LIFO.

Checking with One Stack

Algorithm

```
for each character ch in the string:
    if ch is an opener: push(ch)
    if ch is a closer:
        if stack empty or top is not its match: report unbalanced
        else pop()
at the end: balanced iff stack is empty
```

- Time $O(n)$ worst case, auxiliary space $O(n)$ worst case.

Trace: ([]) vs. ([])

next	stack after step	note
(	(	push opener
[	([	push opener
]	(	matches [: pop
)	empty	matches (: pop — balanced
(	(	push opener
[	([	push opener
)	—	top is [: unbalanced

- Success empties the stack; the first mismatch stops the scan.

Try it on { (() } [before the lab — where does it fail?

Application 2: Evaluating Postfix

Postfix in one line

The operator follows its operands: $2\ 3\ 4\ +\ *$ means $2 \times (3 + 4)$. No parentheses, no precedence rules.

- ▶ Infix ($2 * (3 + 4)$) is for humans; postfix is for stacks.
- ▶ Full conversion between the forms is Week 4's job — today we only *evaluate* a postfix string that is already given.

Standard treatment (see Horowitz & Sahni Ch. 3; Aho et al. Ch. 2) [2, 1].

Evaluating with One Stack (Lab P2)

Algorithm

```
for each token t in the postfix string:
    if t is a number: push(t)
    if t is an operator op:
        b = pop(); a = pop()
        push(apply(op, a, b))
at the end: answer is pop()
```

- ▶ Time $O(n)$ worst case, auxiliary space $O(n)$ worst case.
- ▶ Order matters: b is popped first, a second.

Trace: 2 3 4 + *

token	stack after step
2	2
3	2 3
4	2 3 4
+	2 7
*	14

- ▶ + pops 4 then 3, pushes 7; * pops 7 then 2, pushes 14.
- ▶ One left-to-right pass, no backtracking — the stack remembers the pending operands.

Two applications, one pattern: the stack remembers what is pending.

Three Common Mistakes

- ▶ **Starting indices at 1 in C code** — `A[1..n]` alongside C silently drops `A[0]` and reads past the end.
- ▶ **Forgetting `top == -1` means empty** — then `pop` reads garbage instead of reporting underflow.
- ▶ **Swapping row- and column-major** — 1024 vs. 1028 in our example; same indices, different bytes.

The habit

Name the operation, give the bound, state the case — and point at the index or trace that exhibits it.

Week 3 Summary

- ▶ **Arrays**: contiguous cells; $\text{addr}(A[i]) = \text{base} + i \times w$; 2D row-major adds $(i \times c) + j$.
- ▶ **Costs**: access $O(1)$; search and mid-insert $O(n)$ — all worst case.
- ▶ **Stack ADT**: push/pop/peek, LIFO; array implementation with top ($-1 = \text{empty}$), each $O(1)$.
- ▶ **Applications**: bracket checking and postfix evaluation, each one pass at $O(n)$ time.

Next week

From postfix to prefix and back, Hanoi, and recursion with the runtime stack — the stack starts calling functions, not just holding numbers.

Before You Go: Predict

Three one-line predictions

Answer from the formulas, then check in the lab.

1. A 4×5 int array ($w = 4$, $\text{base} = 5000$): row-major address of $A[2][3]$?
2. `push(5)`, `push(7)`, `pop()`, `push(9)`: final `top` and `peek()`?
3. Is $\{[(\)]\}$ balanced? Which step first disagrees?

Answers: 5052; `top` = 1, `peek()` = 9; no — the (meets] .

References I



Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman.

Data Structures and Algorithms.

Addison-Wesley, 1st edition, 1983.



Ellis Horowitz and Sartaj Sahni.

Fundamentals of Data Structures.

Galgotia Booksource, 2nd edition, 2008.



Narasimha Karumanchi.

Data Structures And Algorithms Made Easy.

CareerMonk Publications, 2017.

This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4th edition, 2011.

References II



Niklaus Wirth.

Algorithms and Data Structures.

Author (freely distributed), 2004.

Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.